



SATHYABAMA

**INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)**

Chennai - 600119

MAIN PROJECT

UNDER THE GUIDANCE OF :
MS. MEHVEESH MARIYA, ASSISTANT
PROFESSOR,
DEPARTMENT OF COMPUTER SCIENCE
SIST.

NAME : Ajay Deepak.J, Ditto.D
REG NO : 42290005, 42290026
DEPT : Bsc.Computer Science(3rd year).

TITLE:

COMMUNICATION BARRIER NULLIFIER USING MACHINE LEARNING

GESTURE TO VOICE TO WORDS

ABSTRACT:

- Sign language recognition has been widely used for communication amongst the hearingimpaired, robotic communication and non-verbal community. This project focuses on developing a real-time sign language to text and voice conversion , then text to gesture

conversion system using machine learning, aimed at improving communication for the deaf and mute community.

- **Mediapipe** is utilized for detecting and tracking hand landmarks, identifying 21 key points on each hand that are essential for recognizing various gestures.
- Once the hand landmarks are detected, features are extracted from these points and used as inputs for machine learning models.
- The system incorporates **AdaBoost**, which combines multiple weak classifiers into a strong classifier, **Random Forest** for creating an ensemble of decision trees to classify gestures, and **Gradient Boosting** to iteratively enhance model performance by correcting errors from previous models.
- Additionally, a **Hybrid Stacking Classifier** is used to combine the predictions from these models, improving overall accuracy by leveraging a meta-model that optimizes the integration of base model predictions.

INTRODUCTION:

Sign language is a vital mode of communication for the deaf and mute community, enabling them to express themselves and interact with

others.However, bridging the communication gap between sign language users and those who do not understand it remains a significant challengeTraditional methods, such as interpreters, are often not readily available, and relying on them can limit spontaneous and fluid interactions.ML algorithms can analyze and learn from vast amounts of gesture data, enabling real-time recognition and translation with high accuracy.

LITERATURE SURVEY:

S.NO	PROJECT TITLE	AUTHOR	YEAR	METHODOLOGY	RESULT
1	Sign Language to Text Translation Using Convolutional Neural Network	Trivedi Devarsh Gunvantray; T Ananthan	2024	Convolutional Neural Network (CNN)	employs 24 classes, representing the letters A to Y

2	Sign Language to Text Translation with Computer Vision: Bridging the Communication Gap	So Xue Thong; Eng Lip Tan; Ching Pang Goh	2024	utilizes Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM)	machine translation and text-to-speech
3	Sign Language to Text Conversion using RNN-LSTM	Amutha Seviappan; Kothai Ganesan; Anna Anbumozhi	2023	LSTM networks	Sign language was converted to text format

S.NO	PROJECT TITLE	AUTHOUR	YEAR	METHODOLOGY	RESULT
4	Sign Language To Text Translator: A Semantic Approach With Ontological Framework	S Asad Ahmed; Y Jabir Sheriff; B Prakash	2024	Ontological framework	captured hand gestures and convert them into text

5	Indian Sign Language Recognition with Conversion to Bilingual Text	Nidhi Chandarana; Shreya Manjucha; Priyansi Chogale; Nidhish Chhajed	2023	SVM, KNN, Random Forest, and Decision Tree classifiers;	result in the text as well as audio format
6	Hand Talk: Sign Language to Text Converter using CNN	N. Shirisha; D. B. V. Jagannadham; Padma Parshapu	2024	Convolutional Neural Network (CNN) classifier	improve the quality of life for individuals with hearing impairments

EXISTING SYSTEM:

The ontological framework is designed for capturing hand gestures and converting them into text. This framework employs single machine learning techniques specifically tailored for recognizing Indian Sign Language, enabling the conversion of gestures into text and audio

formats. A Convolutional Neural Network (CNN) classifier plays a crucial role in classifying gestures into corresponding text representations. Additionally, the integration of Convolutional Neural Networks (CNN) with Long Short-Term Memory (LSTM) networks enhances the translation process from sign language to text, offering a more accurate and efficient solution.

DISADVANTAGES:

- **Sign Language to Text Conversion:**
- Most models focus on converting sign language gestures to text, limiting versatility for other forms like speech translation.
- **Challenges in Vision-Based Systems:**
- Struggle with **varying lighting conditions**, such as bright or dim environments.
- **Complex backgrounds** can interfere with hand gesture recognition, reducing accuracy.
- **Real-Time Translation Issues:**
- Accuracy is often compromised due to the need for fast processing and subtle gesture variations.

- Limited datasets and computational inefficiencies contribute to errors in real-time scenarios.
- **Dependence on Single Machine Learning Models:**
- Single models lack flexibility to handle diverse sign language dialects or gestures.
- Performance drops in uncontrolled environments due to overfitting or lack of adaptability.
- Using ensemble or hybrid models can improve robustness and overall accuracy.

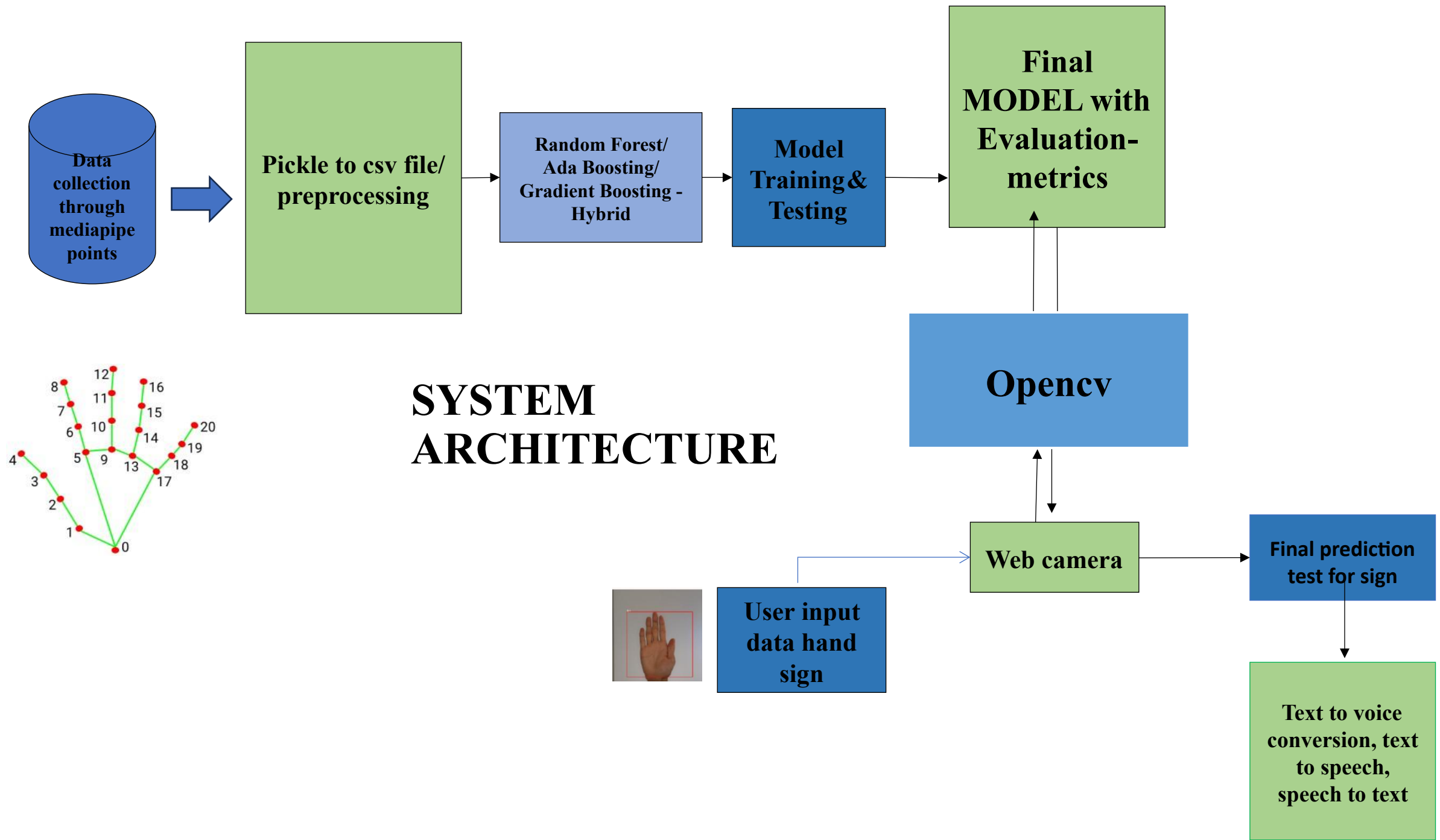
PROPOSED SYSTEM:

The proposed system begins with Hand Landmark Detection, utilizing Mediapipe to detect and track 21 key points on each hand, which are crucial for recognizing various gestures accurately. Once the landmarks are detected, the Feature Extraction process extracts meaningful data from these key points, serving as inputs for the machine learning models. To classify gestures, the system implements multiple Machine Learning Models, including AdaBoost, Random Forest, and Gradient Boosting, each contributing to recognizing sign language gestures effectively. To further enhance accuracy, a Hybrid Stacking Classifier is employed, integrating predictions from the individual models using a meta-model, which optimizes the combination of classifiers for improved performance. Finally, the Output Module facilitates bidirectional communication, converting recognized sign language gestures into text and speech, ensuring accessibility for both

hearing-impaired and non-sign language users. The incorporation of text-to-speech conversion enhances interaction, making the system more versatile and user-friendly.

ADVANTAGES:

- **Real-Time Translation:** Converts sign language gestures into text and speech instantly for seamless communication.
- **Bidirectional Communication:** Supports both sign-to-text/speech and text-to-sign translation for effective interaction.
- **High Accuracy:** Uses AdaBoost, Random Forest, Gradient Boosting, and Hybrid Stacking Classifier for precise recognition.
- **Robust Hand Tracking:** Mediapipe detects and tracks 21 key hand landmarks for accurate gesture recognition.
- **Feature Extraction:** Captures essential motion and positional data for improved classification.
- **Multimodal Output:** Provides both text and speech output to enhance accessibility.
- **Adaptability:** Works across different lighting conditions and complex backgrounds.



SYSTEM REQUIREMENTS:

HARDWARE:

- Processor: Intel® Core™ i3-2350M CPU @ 2.30GHz Installed memory (RAM):4.00GB
- System Type: 64-bit Operating System
- Hard disk: 10 GB of available space or more.
- Display: Dual XGA (1024 x 768) or higher resolution monitors
- Operating system: Windows

SOFTWARE

- PYCHARM IDE
- ANACONDA

MODULES:

- DATA COLLECTION

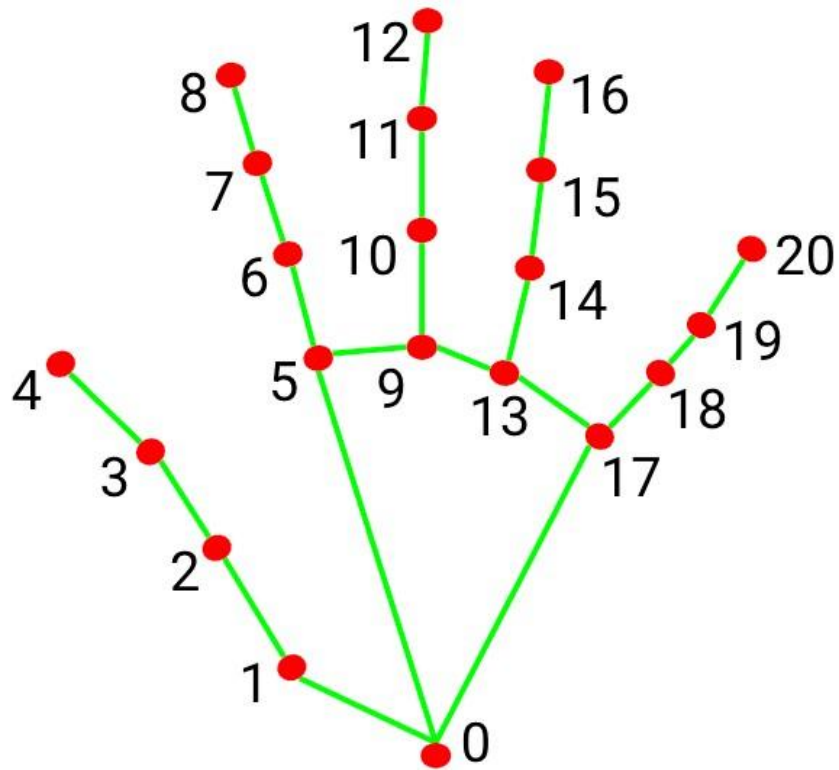
- PREPROCESSING
- MODEL TRAINING
- MODEL TESTING & PERFORMANCE EVALUATION
- GESTURE TO SPEECH CONVERSION
- TEXT TO SPEECH
- SPEECH TO TEXT

DATA COLLECTION PROCEDURE:

- Sign language recognition module: A module that recognizes the sign language gestures performed by the user and create a numerical value.
- **Data Acquisition** – Collects data from cameras, sensors, or public datasets, ensuring diverse sources and multiple conditions for accuracy.
- **Annotation & Labeling** – Manually or automatically labels gestures to enable supervised learning and improve recognition.

- **Preprocessing & Splitting** – Applies noise reduction, normalization, and augmentation before dividing into training, validation, and testing sets.
- **Ethical Considerations** – Ensures user consent, data privacy, and compliance with security regulations for responsible data usage.

DATA COLLECTION PROCEDURE:



- | | |
|-----------------------|-----------------------|
| 0. WRIST | 11. MIDDLE_FINGER_DIP |
| 1. THUMB_CMC | 12. MIDDLE_FINGER_TIP |
| 2. THUMB_MCP | 13. RING_FINGER_MCP |
| 3. THUMB_IP | 14. RING_FINGER_PIP |
| 4. THUMB_TIP | 15. RING_FINGER_DIP |
| 5. INDEX_FINGER_MCP | 16. RING_FINGER_TIP |
| 6. INDEX_FINGER_PIP | 17. PINKY_MCP |
| 7. INDEX_FINGER_DIP | 18. PINKY_PIP |
| 8. INDEX_FINGER_TIP | 19. PINKY_DIP |
| 9. MIDDLE_FINGER_MCP | 20. PINKY_TIP |
| 10. MIDDLE_FINGER_PIP | |

1	Letter,unit-0,unit-1,unit-2,unit-3,unit-4,unit-5,unit-6,unit-7,unit-8,unit-9,unit-10,unit-11,unit-12,unit-13,unit-14,unit-15,unit-16,unit-17,unit-18,unit-19,unit-20,unit-21,unit-22,unit-23,unit-24,unit-25,unit-26,unit-27,unit-28,unit-29,unit-30,unit-31,unit-32,unit-33,unit-34,unit-35,unit-36,unit-37,unit-38,unit-39,unit-40,unit-41,unit-42,unit-43,unit-44,unit-45,unit-46,unit-47,unit-48,unit-49,unit-50,unit-51,unit-52,unit-53,unit-54,unit-55,unit-56,unit-57,unit-58,unit-59,unit-60,unit-61,unit-62,unit-63,unit-64,unit-65,unit-66,unit-67,unit-68,unit-69,unit-70,unit-71,unit-72,unit-73,unit-74,unit-75,unit-76,unit-77,unit-78,unit-79,unit-80,unit-81,unit-82,unit-83,unit-84,unit-85,unit-86,unit-87,unit-88,unit-89,unit-90,unit-91,unit-92,unit-93,unit-94,unit-95,unit-96,unit-97,unit-98,unit-99	✓
2	Play Music,0.503330545,0.256193347,0.228518209,0.215572536,0.228518209,0.414835321,0.315218675,0.443037559,0.418972286,0.302340794,0.319070701,0.289645008,0.324574441,0.285741814,0.311552401,0.289880283,0.318947191,0.297362956,0.313138531,0.292964957,0.316470701,0.31283877,0.426421367,0.406437415,0.285326647,0.31283877,0.422649678,0.402614666,0.290276991,0.321463201,0.279787831,0.309839481,0.282673662,0.307097954,0.287978577,0.31048581,0.284853222,0.30358091,0.287007509,0.294338491,0.391884598,0.277104257,0.303660588,0.290194204,0.298063701,0.281536155,0.311416331,0.289437931,0.31187281,0.286427625,0.302210839,0.271163072,0.306997701,0.282211079,0.303782001,0.288210338,0.300437101,0.274019717,0.304195201,0.278247571,0.311969441,0.281873775,0.316035201,0.289329824,0.324394797,0.289645008,0.324574441,0.285741814,0.311552401,0.289880283,0.318947191,0.297362956,0.313138531,0.292964957,0.316470701,0.31283877,0.426421367,0.406437415,0.285326647,0.31283877,0.422649678,0.402614666,0.290276991,0.321463201,0.279787831,0.309839481,0.282673662,0.307097954,0.287978577,0.31048581,0.284853222,0.30358091,0.287007509,0.294338491,0.391884598,0.277104257,0.303660588,0.290194204,0.298063701,0.281536155,0.311416331,0.289437931,0.31187281,0.286427625,0.302210839,0.271163072,0.306997701,0.282211079,0.303782001,0.288210338,0.300437101,0.274019717,0.304195201,0.278247571,0.311969441,0.281873775,0.316035201,0.289329824,0.324394797,0.289645008,0.324574441,0.285741814,0.311552401,0.289880283,0.318947191,0.297362956,0.313138531,0.292964957,0.316470701,0.31283877,0.426421367,0.406437415,0.285326647,0.31283877,0.422649678,0.402614666,0.290276991,0.321463201,0.279787831,0.309839481,0.282673662,0.307097954,0.287978577,0.31048581,0.284853222,0.30358091,0.287007509,0.294338491,0.391884598,0.277104257,0.303660588,0.290194204,0.298063701,0.281536155,0.311416331,0.289437931,0.31187281,0.286427625,0.302210839,0.271163072,0.306997701,0.282211079,0.303782001,0.288210338,0.300437101,0.274019717,0.304195201,0.278247571,0.311969441,0.281873775,0.316035201,0.289329824,0.324394797,0.289645008,0.324574441,0.285741814,0.311552401,0.289880283,0.318947191,0.297362956,0.313138531,0.292964957,0.316470701,0.31283877,0.426421367,0.406437415,0.285326647,0.31283877,0.422649678,0.402614666,0.290276991,0.321463201,0.279787831,0.309839481,0.282673662,0.307097954,0.287978577,0.31048581,0.284853222,0.30358091,0.287007509,0.294338491,0.391884598,0.277104257,0.303660588,0.290194204,0.298063701,0.281536155,0.311416331,0.289437931,0.31187281,0.286427625,0.302210839,0.271163072,0.306997701,0.282211079,0.303782001,0.288210338,0.300437101,0.274019717,0.304195201,0.278247571,0.311969441,0.281873775,0.316035201,0.289329824,0.324394797,0.289645008,0.324574441,0.285741814,0.311552401,0.289880283,0.318947191,0.297362956,0.313138531,0.292964957,0.316470701,0.31283877,0.426421367,0.406437415,0.285326647,0.31283877,0.422649678,0.402614666,0.290276991,0.321463201,0.279787831,0.309839481,0.282673662,0.307097954,0.287978577,0.31048581,0.284853222,0.30358091,0.287007509,0.294338491,0.391884598,0.277104257,0.303660588,0.290194204,0.298063701,0.281536155,0.311416331,0.289437931,0.31187281,0.286427625,0.302210839,0.271163072,0.306997701,0.282211079,0.303782001,0.288210338,0.300437101,0.274019717,0.304195201,0.278247571,0.311969441,0.281873775,0.316035201,0.289329824,0.324394797,0.289645008,0.324574441,0.285741814,0.311552401,0.289880283,0.318947191,0.297362956,0.313138531,0.292964957,0.316470701,0.31283877,0.426421367,0.406437415,0.285326647,0.31283877,0.422649678,0.402614666,0.290276991,0.	

PREPROCESSING MODULE:

- Preprocessing is the **data refinement process** that prepares raw data for machine learning models by improving its quality, consistency, and usability.
- Preprocessing ensures that the data fed into machine learning models is **clean, structured, and optimized** for accurate and reliable predictions.

STEPS IN PREPROCESSING: •

Checking the units of data.

- Assign dependent variable and independent variable.
- splitting the data into training data and testing data.
- Checking for missing values

MODEL TRAINING:

- Model training is the process of teaching a machine learning model to recognize patterns and make predictions based on input data.
- **ALGORITHMS USED:**
- **Random Forest:** It's an ensemble learning method that creates multiple decision trees during training and outputs the most frequent prediction (classification) or average (regression) from all trees. It helps improve accuracy and reduce overfitting.
- **SVM (Support Vector Machine):** A supervised learning algorithm that finds the hyperplane which best separates different classes in the feature space, maximizing the margin between classes. It's effective for both classification and regression.
- **K-Nearest Neighbors (KNN):** A simple, instance-based learning algorithm where classification is based on the majority label of the 'K' closest data points to the input. It's non-parametric and works well for classification tasks with well-defined boundaries.

PERFORMANCE EVALUATION:

Model testing refers to the process of evaluating a machine learning model's performance using unseen data (often called the test set) to assess how well the model generalizes to new, real-world data.

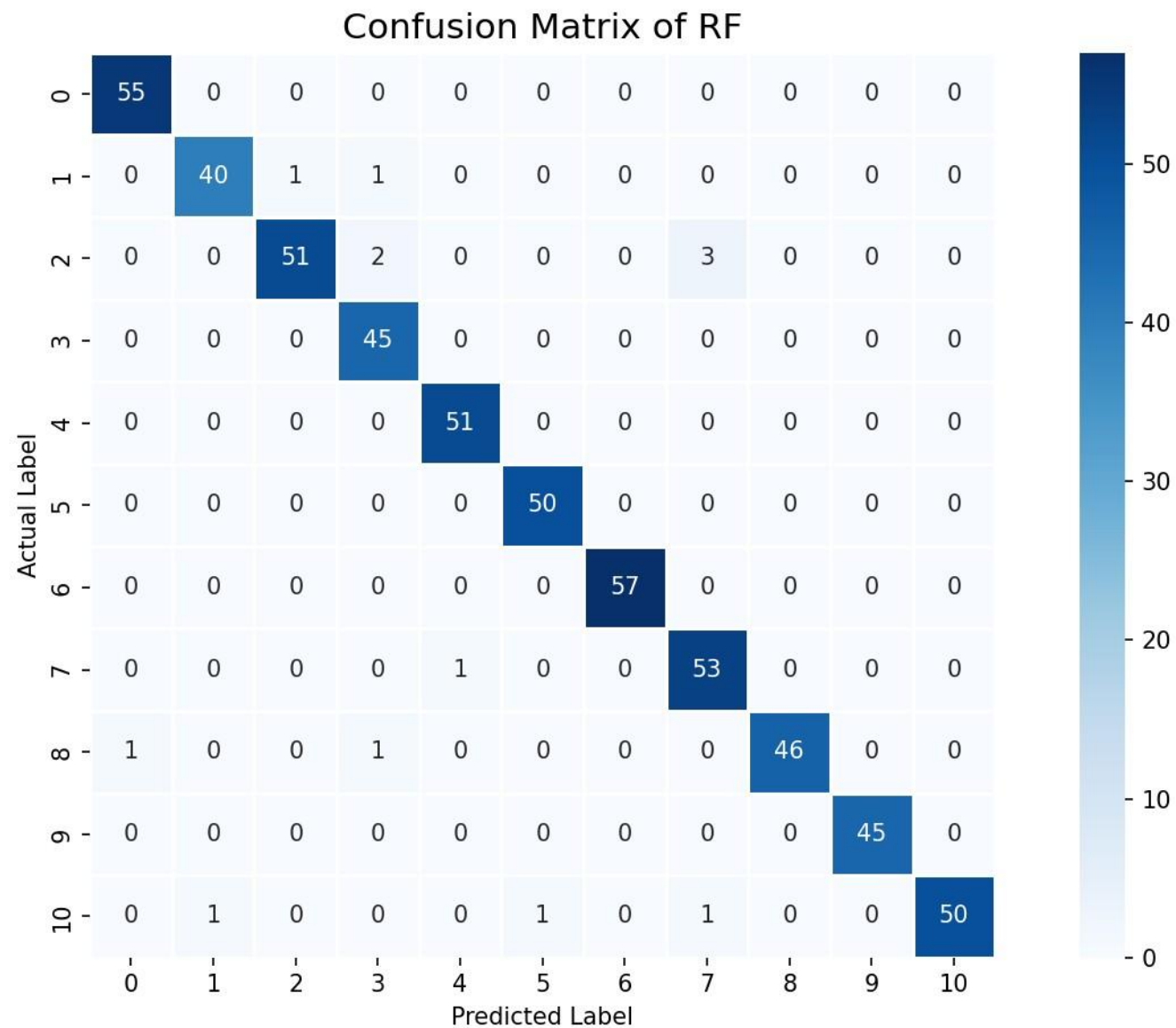
PRECISION:

- precision is calculated as the number of true positives divided by the total number of true positives and false positives.
- $\text{Precision} = \text{TruePositives} / (\text{TruePositives} + \text{FalsePositives})$

ACCURACY:

- Accuracy represents the number of correctly classified data instances over the total number of data instances.

$$\text{Accuracy} = \frac{TN + TP}{TN + FP + TP + FN}$$



TEXT TO SPEECH:

- The **Text-to-Speech (TTS)** system converts text input into spoken words using a speech synthesis engine. The script we provided uses the `pyttsx3` library, which is an offline text-to-speech conversion library.
- **How it Works:**
- **Initialize the speech engine** → `pyttsx3.init()` creates an instance of the text-to-speech engine.
- **Set properties:**
 - **rate:** Controls the speed of speech (default is around 200 words per minute).
 - **volume:** Adjusts the loudness (ranges from 0.0 to 1.0).
- **Take user input** → The script prompts the user to enter text via the console.
- **Convert text to speech** → `engine.say(text)` sends the input text to the speech engine.
- **Speak the text** → `engine.runAndWait()` processes the text and speaks it out loud.

SPEECH TO TEXT:

- The **Speech-to-Text (STT)** system converts spoken words into written text using Automatic Speech Recognition (ASR). Our script utilizes the `speech_recognition` library,
- **How it Works:**
- **Initialize the recognizer** → `sr.Recognizer()` creates an instance of the speech recognition engine.
- **Capture audio:**
 - `sr.Microphone()` accesses the system's microphone.
 - `recognizer.adjust_for_ambient_noise(source)` filters background noise.
 - `recognizer.listen(source)` records user speech.
- **Convert speech to text:**
 - `recognizer.recognize_google(audio)` uses **Google's Speech API** to transcribe speech.
- **Display the result:**
 - If successful, it prints the recognized text.

SAMPLE CODE:

```
import speech_recognition as sr
```

```
def speech_to_text():
```

```
    recognizer = sr.Recognizer() with sr.Microphone() as source: print("Speak  
something...") recognizer.adjust_for_ambient_noise(source) # Adjusts for  
background noise audio = recognizer.listen(source)
```

```
    try:
```

```
        text = recognizer.recognize_google(audio) # Convert speech to text using Google Speech Recognition print("You  
said:", text)
```

```
    except sr.UnknownValueError:
```

```
        print("Sorry, could not understand the audio.")
```

```
    except sr.RequestError:
```

```
        print("Error connecting to speech recognition service.")
```

```
if __name__ == "__main__":
```

```
    speech_to_text()
```

MODEL TRINING:-

```

#####Initialise    random    forest    local_path    =

(os.path.dirname(os.path.realpath('__file__')))

file_name = ('a data.csv') # file of total data
data_path = os.path.join(local_path, file_name)
print(data_path) df = pd.read_csv(r" +
data_path) print(df) units_in_data = 28 # no.
of units in data

titles = [] for i in
range(units_in_data):
titles.append("unit-" + str(i)) X
= df[titles] y = df['letter']
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.5, random_state=2) from sklearn.metrics
import    accuracy_score,    classification_report,
confusion_matrix

clf = RandomForestClassifier(n_estimators=30) # random
forest clf.fit(X_train, y_train) y_pred = clf.predict(X_test)

```

```
print('Accuracy: ', metrics.accuracy_score(y_test, y_pred))
cmrf = confusion_matrix(y_test, y_pred) print("1.Random
Forest Accuracy")
```

```
print("Random Forest classification_report")
print(classification_report(y_pred, y_test, labels=None))
print("Random Forest confusion_matrix")
print(cmrf) import matplotlib.pyplot as plt
import seaborn as sns from sklearn.metrics
import confusion_matrix cm =
confusion_matrix(y_test, y_pred)
print("CONFUSION MATRIX OF RF")
print(cm) tpr = float(cm[0][0]) /
np.sum(cm[0]) fpr = float(cm[1][1]) /
np.sum(cm[1]) plt.figure(figsize=(12, 12))
sns.heatmap(cm, annot=True, fmt=".0f", linewidths=.5,
square=True, cmap='Blues'); plt.ylabel('Actual label');
plt.xlabel('Predicted label'); all_sample_title =
'Confusion Matrix of RF ' plt.title(all_sample_title,
size=15); plt.show()
```

```
clf1 = KNeighborsClassifier() # random forest
clf1.fit(X_train, y_train)
y_pred = clf1.predict(X_test)
print('Accuracy: ', metrics.accuracy_score(y_test, y_pred))
cmrf = confusion_matrix(y_test, y_pred) print("knn
Accuracy") print("knn classification_report")
print(classification_report(y_pred, y_test, labels=None))
```

```
import matplotlib.pyplot as plt import
seaborn as sns from sklearn.metrics import
confusion_matrix
```

```
cm = confusion_matrix(y_test, y_pred)
print("CONFUSION MATRIX OF knn")
print(cm) tpr = float(cm[0][0]) /
np.sum(cm[0]) fpr = float(cm[1][1]) /
np.sum(cm[1]) plt.figure(figsize=(12,
12))
sns.heatmap(cm, annot=True, fmt=".0f", linewidths=.5,
square=True, cmap='Blues'); plt.ylabel('Actual label');
```



```
plt.xlabel('Predicted label'); all_sample_title =  
'Confusion Matrix of knn ' plt.title(all_sample_title,  
size=15); plt.show() from sklearn.svm import SVC  
clf2 = SVC() # random forest  
clf2.fit(X_train, y_train)  
y_pred = clf2.predict(X_test)  
print('Accuracy: ', metrics.accuracy_score(y_test, y_pred))  
cmsvc = confusion_matrix(y_test, y_pred) print("1.svm  
Accuracy")
```

```
print("svm classification_report")  
print(classification_report(y_pred, y_test, labels=None))  
print("svm confusion_matrix") print(cmrf)  
import matplotlib.pyplot as plt import  
seaborn as sns from sklearn.metrics import  
confusion_matrix cm =  
confusion_matrix(y_test, y_pred)
```

```
tpr = float(cm[0][0]) / np.sum(cm[0])  
fpr = float(cm[1][1]) / np.sum(cm[1])  
plt.figure(figsize=(12, 12))
```

```
sns.heatmap(cm, annot=True, fmt=".0f", linewidths=.5,  
square=True, cmap='Blues');  
plt.ylabel('Actual label'); plt.xlabel('Predicted  
label'); all_sample_title = 'Confusion Matrix  
of svm ' plt.title(all_sample_title, size=15);  
plt.show()
```

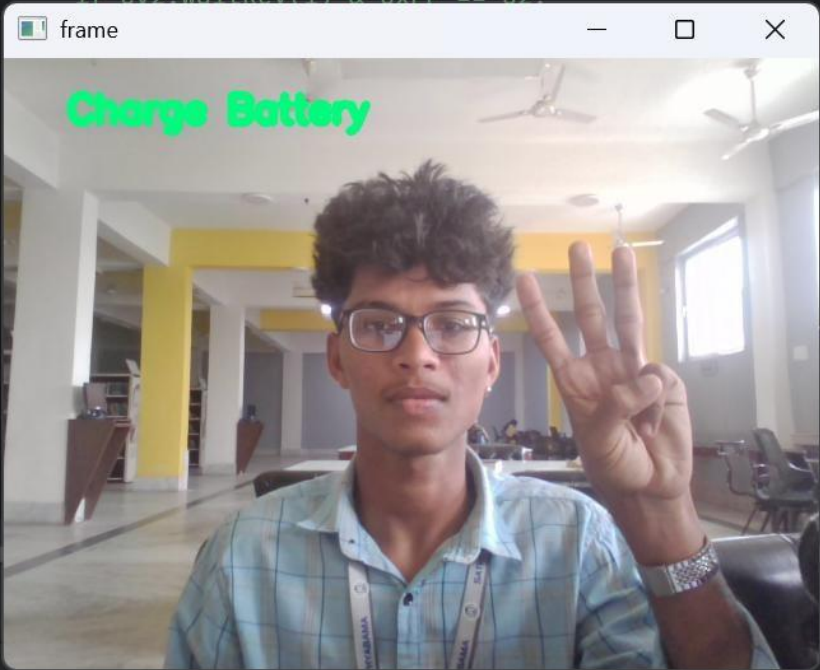
Project

- voice_ C:\Users\Ajay deepak.J\Pycharm
- .venv library root
- a data.csv
- gesture to speech.py
- gesture to text.py
- HandDataCollector.py
- model.py
- speech to txt.py
- text to speech.py
- External Libraries
- Scratches and Consoles

a data.csv model.py gesture to text.py text to speech.py speech to txt.py gesture to sp

```
image = ImageData['image']  
  
if cv2.waitKey(1) & 0xFF == 32:
```

frame



2, 255)
v2.FONT_HERSHEY_SIMPLEX, fontScale: 1, color:

Run gesture to speech (1)

```
W0000 00:00:1741319570.231050 22188 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Di  
W0000 00:00:1741319570.248677 22396 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Di  
C:\Users\Ajay deepak.J\PycharmProjects\voice\.venv\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid featur  
warnings.warn(  
W0000 00:00:1741319570.371381 6292 inference_feedback_manager.cc:114] Feedback manager requires a  
W0000 00:00:1741319570.400279 7720 inference_feedback_manager.cc:114] Feedback manager requires a  
C:\Users\Ajay deepak.J\PycharmProjects\voice\.venv\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid featur  
warnings.warn(  
W0000 00:00:1741319570.516690 17760 inference_feedback_manager.cc:114] Feedback manager requires a  
W0000 00:00:1741319570.539600 13116 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Di
```

Microsoft Defender may affect IDE
To avoid performance issues, exclude the IDE and the project folders from the...
Exclude folders More

voice_ > gesture to text.py 102:1 CRLF UTF-8 4 spaces Python 3.12 (voice_)

voice_ Version control

Project

voice_ C:\Users\Ajay deepak.J\Pycharm

.venv library root

a data.csv

gesture to speech.py

gesture to text.py

HandDataCollector.py

model.py

speech to txt.py

text to speech.py

External Libraries

Scratches and Consoles

gesture to speech (1)

model.py

gesture to text.py

text to speech.py

speech to txt.py

gesture to sp

5 7 3

102:1 CRLF UTF-8 4 spaces Python 3.12 (voice_)

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

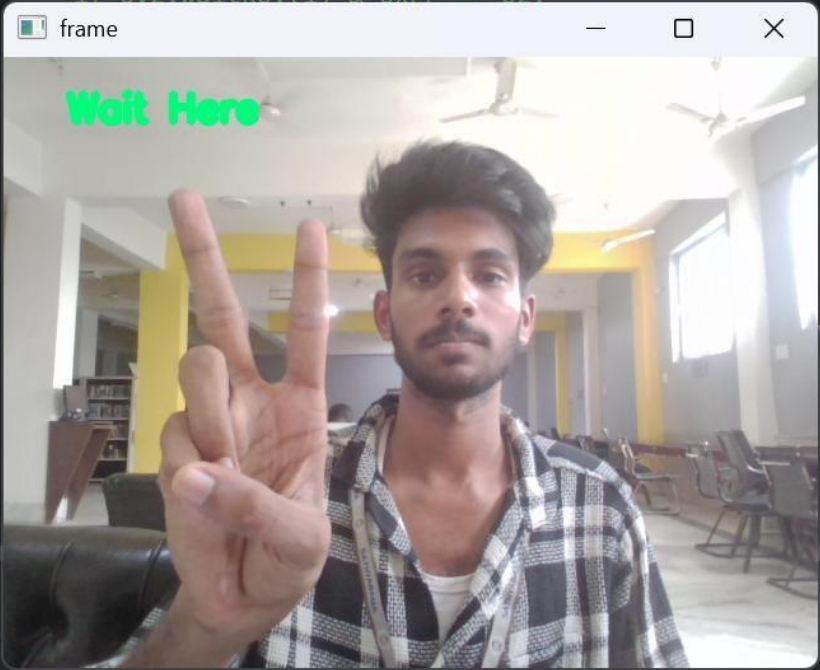
102

image = ImageData['image']

if cv2.waitKey(1) & 0xFF == 32:

frame

Wait Here



2, 255)

cv2.FONT_HERSHEY_SIMPLEX, fontScale: 1, color:

Run

gesture to speech (1)

W0000 00:00:1741319740.517400 20096 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Di

W0000 00:00:1741319740.534142 20096 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Di

C:\Users\Ajay deepak.J\PycharmProjects\voice\.venv\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid featu

warnings.warn(

W0000 00:00:1741319740.636490 17632 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Di

W0000 00:00:1741319740.652584 17632 inference_feedback_manager.cc:114] Feedback manager requires a model with a single signature inference. Di

C:\Users\Ajay deepak.J\PycharmProjects\voice\.venv\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid featu

warnings.warn(

Future work :

1. Improved Models for Gesture and Sign Recognition

- Multimodal Recognition:** Current systems often focus on either gesture recognition (hand movements) or facial expressions. Future work could integrate both, as facial expressions play a significant role in sign language (e.g., for tonal or grammatical variations).
- 3D Convolutional Neural Networks (3D CNNs):** Moving beyond 2D image recognition to 3D CNNs could help capture depth and movement more accurately, allowing systems to better interpret dynamic hand gestures and positions in space.

2. Improving Real-Time Recognition

- Low-Latency Systems:** Real-time recognition remains a challenge. Future systems could utilize more efficient algorithms or specialized hardware (e.g., **Edge AI, FPGA, GPU accelerators**) to reduce the time delay between input and output.
- On-Device Recognition:** With advancements in **edge computing**, sign language recognition could be done directly on smartphones, wearables, or other devices, reducing the need for cloud processing and improving responsiveness.

3. Cross-Domain and Cross-Language Systems

- Domain Adaptation:** Most existing SLR systems are tailored to a specific domain or dataset. Future systems could use transfer learning to recognize signs from different contexts or applications (e.g., medical, legal, educational domains).

- Multilingual and Cross-Sign-Language Systems:** There are many different sign languages across the world, and each has its unique syntax, semantics, and gestures. A system that can understand multiple sign languages (e.g., ASL, BSL, ISL) or translate between them could significantly improve global accessibility.

4. Collaborative Human-AI Interaction

- Personalized Learning:** Future systems could incorporate **adaptive learning** techniques to improve based on user feedback, ensuring that the system continually evolves to suit the unique needs of individual users over time.

- Collaborative AI:** Rather than replacing human interpreters or sign language users, AI systems could collaborate with them, assisting in communication while also allowing the users to guide the system.

Conclusion:

Improving communication for the Deaf community. Advancements in **multimodal systems** and **real-time recognition**

will enhance the accuracy and fluidity of interactions. Cross-lingual models can help bridge gaps between different

sign languages globally, promoting inclusivity. **Diverse and large-scale datasets** will be essential for building more accurate models, while **synthetic data generation** can help overcome dataset limitations. The integration of

AR/VR

will enable immersive learning and real-time translations. Ensuring **ethical AI** and **privacy protection** is crucial as

these systems evolve. **Explainable AI** will foster trust, especially in sensitive environments like healthcare.

The future of SLR aims to provide a **collaborative, human-centered experience**, breaking down communication barriers. By focusing on **personalized recognition** and **inclusive design**, SLR can empower the Deaf community and foster global accessibility. The ultimate goal is to create a world where technology enables seamless communication for all.

THANK YOU.